Mireya Alvarez
Yu-Fong
Nathalie Valle
Pablo Fuentes

# Documentación del modelo de datos

## Sistema de gestión de restaurante — Proyecto 2, Base de Datos 1

### 1. Objetivo y alcance

La base de datos permite administrar las sucursales Oakland Mall y Ciudad Cayalá. Almacena información sobre clientes, mesas, reservas, visitas, preferencias, productos, recetas, inventario, ventas, usuarios y auditoría.

El sistema utiliza PostgreSQL como almacenamiento principal y Python para la interacción mediante consola.

El modelo contiene 16 tablas y 67 columnas. Las sucursales comparten el catálogo de productos, ingredientes, recetas y clientes. Las mesas, reservas, visitas y existencias se relacionan con una sucursal específica.

Las entradas de inventario se registran directamente mediante lotes. No se incluyen compras ni proveedores.

### 2. Organización y relaciones principales

#### Sucursales y mesas

Una sucursal puede tener muchas mesas. Cada mesa pertenece a una sola sucursal y su número no puede repetirse dentro de ella.

#### Clientes y reservas

Un cliente puede realizar muchas reservas. Cada reserva identifica a un cliente principal y pertenece a una sucursal.

Una reserva puede utilizar varias mesas y una mesa puede participar en distintas reservas a lo largo del tiempo. La tabla `reserva_mesa` representa esta relación.

#### Clientes y visitas

Un cliente puede realizar muchas visitas. Cada visita registra su llegada, salida, cantidad de personas y observaciones.

Una visita puede estar asociada a una reserva o corresponder a una llegada sin reserva. Una reserva puede asociarse como máximo con una visita.

Mireya Alvarez
Yu-Fong
Nathalie Valle
Pablo Fuentes
La tabla `visita_mesa` identifica las mesas utilizadas y cuándo se liberan después de la limpieza.

**Productos, preferencias y recetas**

Los productos del menú se almacenan en `plato`, incluyendo las bebidas.

Un cliente puede tener varios productos favoritos y un producto puede ser favorito de varios clientes. Esta relación se registra en `preferencia_cliente`.

Un producto puede necesitar varios insumos y un insumo puede utilizarse en varios productos. La tabla `receta` establece la cantidad de cada insumo necesaria para preparar una unidad del producto.

**Inventario**

Cada lote corresponde a un insumo y una sucursal. Registra recepción, caducidad y cantidad disponible.

La existencia utilizable se calcula sumando los lotes no vencidos de cada insumo en una sucursal.

**Ventas**

Una visita puede tener varias ventas. Cada venta contiene uno o más productos registrados en `detalle_venta`.

El detalle conserva la cantidad y el precio aplicado en el momento de la venta.

**Usuarios y bitácora**

Cada usuario tiene un rol y una sucursal habitual. Un usuario puede estar asociado a muchas entradas de bitácora.

La bitácora permite identificar qué usuario realizó una operación y qué registro fue afectado.

## 3. Convenciones del diccionario

- **PK:** llave primaria que identifica un registro.
- **FK:** llave foránea que referencia un registro de otra tabla.
- **UNIQUE:** impide repetir un valor o combinación.
- **NULL:** permite que el dato esté ausente.
- **IDENTITY:** genera automáticamente el identificador.
- **NUMERIC(p,s):** número decimal con precisión total `p` y `s` posiciones decimales.

Todas las llaves primarias simples utilizan `INTEGER GENERATED ALWAYS AS IDENTITY`.

Los campos son obligatorios, excepto los señalados expresamente como opcionales.

Mireya Alvarez
Yu-Fong
Nathalie Valle
Pablo Fuentes
Las tablas intermedias utilizan llaves primarias compuestas para evitar repetir una misma relación.

## 4. Diccionario de datos

### 4.1. sucursal

Representa una sede del restaurante.

- `id_sucursal` — INTEGER, PK e IDENTITY. Identificador de la sucursal.
- `nombre` — VARCHAR(100). Nombre de la sede.
- `direccion` — VARCHAR(200). Ubicación de la sede.

### 4.2. cliente

Almacena los datos generales de los clientes.

- `id_cliente` — INTEGER, PK e IDENTITY. Identificador del cliente.
- `nombre` — VARCHAR(120). Nombre del cliente.
- `telefono` — VARCHAR(20), opcional. Número telefónico.

Los nombres y teléfonos pueden repetirse. La identidad del cliente se determina mediante `id_cliente`.

### 4.3. mesa

Representa una mesa de una sucursal.

- `id_mesa` — INTEGER, PK e IDENTITY. Identificador general de la mesa.
- `id_sucursal` — INTEGER, FK a `sucursal`. Sucursal a la que pertenece.
- `numero` — INTEGER. Número utilizado para identificarla dentro de la sucursal; debe ser positivo.
- `capacidad` — INTEGER. Cantidad de personas que admite; debe ser positiva.
- `zona` — VARCHAR(10). Acepta `interior` o `exterior`.

La combinación de sucursal y número es única.

`id_mesa` y `numero` tienen propósitos distintos: el primero identifica la fila en la base; el segundo corresponde al número de mesa dentro de la sede.

### 4.4. reserva

Representa una reservación realizada por un cliente.

- `id_reserva` — INTEGER, PK e IDENTITY. Identificador de la reserva.
- `id_cliente` — INTEGER, FK a `cliente`. Cliente principal.

Mireya Alvarez
Yu-Fong
Nathalie Valle
Pablo Fuentes

- `id_sucursal` — INTEGER, FK a `sucursal`. Sede de la reserva.
- `fecha_hora_inicio` — TIMESTAMP. Inicio del horario reservado.
- `fecha_hora_fin_estimada` — TIMESTAMP. Fin estimado; debe ser posterior al inicio.
- `cantidad_personas` — INTEGER. Número de personas; debe ser positivo.
- `estado` — VARCHAR(15). Estado de la reserva.

Estados permitidos:

- `confirmada`
- `cancelada`
- `atendida`
- `no_asistio`

El estado inicial es `confirmada`.

**4.5. reserva_mesa**

Relaciona una reserva con sus mesas.

- `id_reserva` — INTEGER, FK a `reserva`.
- `id_mesa` — INTEGER, FK a `mesa`.

La PK está formada por ambos campos. Una mesa no puede repetirse dentro de la misma reserva.

**4.6. visita**

Representa la asistencia efectiva de un cliente.

- `id_visita` — INTEGER, PK e IDENTITY. Identificador de la visita.
- `id_cliente` — INTEGER, FK a `cliente`. Cliente principal.
- `id_sucursal` — INTEGER, FK a `sucursal`. Sede visitada.
- `id_reserva` — INTEGER, FK a `reserva`, opcional y UNIQUE. Reserva asociada, si existe.
- `fecha_hora_llegada` — TIMESTAMP. Momento de llegada.
- `fecha_hora_salida` — TIMESTAMP, opcional. Momento de salida.
- `cantidad_personas` — INTEGER. Número de asistentes; debe ser positivo.
- `observaciones` — TEXT, opcional. Comentarios sobre la visita o experiencia.

La salida, cuando existe, no puede ser anterior a la llegada.

Cuando se asocia una reserva, debe corresponder al mismo cliente y sucursal.

**4.7. visita_mesa**

Mireya Alvarez
Yu-Fong
Nathalie Valle
Pablo Fuentes
Identifica las mesas utilizadas durante una visita.

- `id_visita` — INTEGER, FK a `visita`.
- `id_mesa` — INTEGER, FK a `mesa`.
- `fecha_hora_liberacion` — TIMESTAMP, opcional. Momento en que la mesa queda disponible después de la limpieza.

La PK está formada por `id_visita` e `id_mesa`.

Una liberación en `NULL` indica que la mesa sigue asignada. Un índice único parcial impide que una mesa tenga dos asignaciones sin liberar.

**4.8. plato**

Almacena los productos ofrecidos, incluyendo alimentos y bebidas.

- `id_plato` — INTEGER, PK e IDENTITY. Identificador del producto.
- `nombre` — VARCHAR(100). Nombre del producto.
- `precio` — NUMERIC(10,2). Precio vigente; debe ser positivo.

El precio del catálogo puede cambiar sin modificar los precios históricos de las ventas.

**4.9. preferencia_cliente**

Registra los productos favoritos de cada cliente.

- `id_cliente` — INTEGER, FK a `cliente`.
- `id_plato` — INTEGER, FK a `plato`.

Ambos campos forman la PK. Un cliente no puede registrar dos veces el mismo favorito.

Las preferencias son compartidas entre sucursales.

**4.10. insumo**

Representa un ingrediente o artículo necesario para vender un producto.

- `id_insumo` — INTEGER, PK e IDENTITY. Identificador del insumo.
- `nombre` — VARCHAR(100). Nombre del ingrediente o artículo.
- `unidad_medida` — VARCHAR(10). Acepta `gramos` o `unidades`.
- `existencia_minima` — NUMERIC(12,3). Mínimo de referencia; no puede ser negativo.

El mínimo se compara con las existencias utilizables de cada sucursal.

La cantidad disponible no se almacena en esta tabla: se calcula a partir de sus lotes.

Mireya Alvarez
Yu-Fong
Nathalie Valle
Pablo Fuentes

**4.11. receta**

Define el consumo de insumos por unidad de producto.

- `id_plato` — INTEGER, FK a `plato`.
- `id_insumo` — INTEGER, FK a `insumo`.
- `cantidad` — NUMERIC(12,3). Cantidad necesaria del insumo; debe ser positiva.

La PK está formada por `id_plato` e `id_insumo`.

La cantidad se interpreta utilizando la unidad de medida del insumo.

**4.12. lote**

Registra una entrada de inventario.

- `id_lote` — INTEGER, PK e IDENTITY. Identificador del lote.
- `id_insumo` — INTEGER, FK a `insumo`. Insumo recibido.
- `id_sucursal` — INTEGER, FK a `sucursal`. Sede donde se almacena.
- `fecha_recepcion` — DATE. Fecha de recepción.
- `fecha_caducidad` — DATE. Fecha de caducidad.
- `cantidad_disponible` — NUMERIC(12,3). Cantidad restante; no puede ser negativa.

La caducidad no puede ser anterior a la recepción.

Un lote vencido se conserva, pero su cantidad no se considera utilizable para ventas.

**4.13. venta**

Representa una operación de venta asociada a una visita.

- `id_venta` — INTEGER, PK e IDENTITY. Identificador de la venta.
- `id_visita` — INTEGER, FK a `visita`. Visita a la que corresponde.
- `fecha_hora` — TIMESTAMP. Momento de registro.

La sucursal se obtiene de la visita asociada.

El total se calcula a partir de sus detalles; no existe una columna de total en esta tabla.

**4.14. detalle_venta**

Contiene los productos y precios aplicados en una venta.

- `id_venta` — INTEGER, FK a `venta`.
- `id_plato` — INTEGER, FK a `plato`.
- `cantidad` — INTEGER. Unidades vendidas; debe ser positiva.

Mireya Alvarez
Yu-Fong
Nathalie Valle
Pablo Fuentes

- `precio_unitario` — NUMERIC(10,2). Precio aplicado; debe ser positivo.

La PK está formada por `id_venta` e `id_plato`.

El subtotal se calcula multiplicando cantidad por precio unitario. El total de la venta es la suma de sus subtotales.

#### 4.15. usuario

Almacena las cuentas de acceso al sistema.

- `id_usuario` — INTEGER, PK e IDENTITY. Identificador de la cuenta.
- `nombre_usuario` — VARCHAR(50), UNIQUE. Nombre de acceso.
- `contrasena_hash` — TEXT. Representación protegida de la contraseña.
- `rol` — VARCHAR(15). Acepta `administrador`, `gerente` o `mesero`.
- `id_sucursal` — INTEGER, FK a `sucursal`. Sucursal habitual.

Incluso el administrador tiene una sucursal habitual, aunque sus permisos permiten operar sobre todas.

Las contraseñas se protegen desde Python mediante PBKDF2-HMAC-SHA256 con sal aleatoria.

#### 4.16. bitacora

Registra automáticamente los cambios de las tablas auditadas.

- `id_bitacora` — INTEGER, PK e IDENTITY. Identificador de la entrada.
- `id_usuario` — INTEGER, FK a `usuario`, opcional. Responsable de la operación.
- `tabla` — VARCHAR(63). Nombre de la tabla afectada.
- `registro` — TEXT. Identificación del registro afectado, representada como un objeto JSON en texto.
- `operacion` — VARCHAR(6). Acepta `INSERT`, `UPDATE` o `DELETE`.
- `fecha_hora` — TIMESTAMP. Utiliza `CURRENT_TIMESTAMP` como valor predeterminado.

El usuario puede ser `NULL` durante la carga SQL o la creación del primer administrador.

La bitácora identifica los registros afectados, pero no guarda una copia completa de sus valores anteriores y nuevos.

## 5. Restricciones de integridad en PostgreSQL

Las llaves primarias identifican los registros y evitan duplicados. Las llaves foráneas impiden referencias a clientes, sucursales, productos u otros registros inexistentes.

Mireya Alvarez
Yu-Fong
Nathalie Valle
Pablo Fuentes

Las restricciones `CHECK` controlan cantidades, precios, estados, zonas, unidades de medida y coherencia básica de fechas.

Los triggers complementan las restricciones para comprobar reglas que involucran varias filas o tablas:

- Las mesas asignadas deben pertenecer a la sucursal correspondiente.
- Las mesas de un mismo grupo deben estar en la misma zona.
- La capacidad conjunta debe ser suficiente.
- Las reservas confirmadas o atendidas no deben superponerse sobre una misma mesa.
- Una visita asociada a una reserva debe conservar el cliente y la sucursal correspondientes.
- La liberación de una mesa debe ser coherente con la salida de la visita.

Las comprobaciones diferidas permiten registrar la reserva o visita y después sus mesas dentro de una misma transacción. La operación completa debe cumplir las reglas al hacer `COMMIT`.

El índice único parcial de `visita_mesa` impide utilizar una mesa en otra visita mientras siga sin liberar.

La salida del cliente y la liberación de la mesa son eventos separados.

## 6. Reglas implementadas en Python

El programa valida credenciales y construye la sesión con el ID, rol y sucursal del usuario.

Los permisos se comprueban en las operaciones, además de controlar las opciones visibles del menú:

- Administrador: todas las sucursales, usuarios, bitácora y reportes.
- Gerente: operaciones de su sucursal, incluyendo inventario.
- Mesero: reservas, visitas y ventas de su sucursal.
- Clientes y preferencias: información compartida.

Las ventas requieren una visita abierta. El sistema obtiene los precios del catálogo, suma los ingredientes necesarios de todos los productos y consume lotes utilizables mediante FEFO: primero los que caducan antes y, en caso de empate, el menor ID.

Las entradas crean lotes nuevos. No se modifican fechas antiguas para renovar inventario vencido.

Las alertas comparan las existencias utilizables con el mínimo establecido y muestran lotes próximos a caducar.

## 7. Transacciones y auditoría

Mireya Alvarez
Yu-Fong
Nathalie Valle
Pablo Fuentes
Cada operación que necesita varios cambios utiliza una sola transacción.

Por ejemplo, una venta incluye su cabecera, detalles y descuento de ingredientes. Una llegada con reserva incluye visita, mesas y actualización del estado de la reserva.

El manejador compartido confirma con `COMMIT` cuando la operación termina correctamente. Si falla, ejecuta `ROLLBACK`.

Los cambios de bitácora participan en la misma transacción que la operación auditada. Si la transacción se revierte, tampoco se conservan esas entradas.

La aplicación comunica el usuario responsable mediante una configuración local de la transacción, utilizada por los triggers de bitácora.

Las transacciones buscan mantener las propiedades ACID:

- Atomicidad: los cambios de una operación se confirman juntos o se revierten.
- Consistencia: se respetan las restricciones de integridad.
- Aislamiento: PostgreSQL administra la interacción entre transacciones; el consumo de lotes utiliza bloqueos de filas.
- Durabilidad: los cambios confirmados permanecen almacenados.

No se desarrollan escenarios adicionales de reservas simultáneas de mesas, de acuerdo con la aclaración del catedrático. Se mantienen las transacciones y las validaciones normales.

## 8. Vistas, consultas e índices

La vista `vw_resumen_historico_sucursales` presenta reservas no canceladas, cantidad de ventas e importe vendido por sucursal.

Agrupa reservas y ventas por separado mediante CTEs para evitar multiplicar cantidades al combinarlas.

La aplicación utiliza esta vista en el reporte acumulado sin fechas. Para períodos específicos utiliza una consulta parametrizada.

Los índices apoyan búsquedas y relaciones. Algunos también participan directamente en la integridad, como el índice único parcial que impide dos asignaciones de mesa sin liberar.

Los reportes calculan sus resultados desde las tablas del sistema; no almacenan totales duplicados en tablas adicionales.

## 9. Decisiones y limitaciones del modelo

- No se almacena la ubicación física exacta de las mesas. El personal determina cuáles pueden juntarse.

Mireya Alvarez
Yu-Fong
Nathalie Valle
Pablo Fuentes

- La disponibilidad para una llegada comprueba el momento actual; no estima si terminará antes de una reserva futura.
- Un lote se considera utilizable si su caducidad es igual o posterior a la fecha actual.
- La alerta del 15 % utiliza las cantidades iniciales como referencia. Es una interpretación documentada del equipo, ya que la guía no define expresamente la base del porcentaje.
- La ventana utilizada para próxima caducidad es de siete días.
- El reporte mensual selecciona lotes por mes de caducidad y muestra existencias actuales. No reconstruye inventario al cierre de meses anteriores.
- La bitácora identifica cambios, pero no constituye un historial completo de cantidades o valores anteriores.
- Los identificadores se generan automáticamente y pueden variar entre instalaciones.

## 10. Implementación y replicación

Los archivos se ejecutan en este orden:

1. `estructura.sql`
2. `restricciones.sql`
3. `indices.sql`
4. `triggers.sql`
5. `datos_iniciales.sql`
6. `vistas_reportes.sql`

La instalación completa se realiza sobre una base nueva y vacía.

El README del repositorio contiene las instrucciones de carga, configuración de Python, creación del primer administrador y uso de los menús.

El diagrama entidad-relación complementa este documento y muestra visualmente las tablas, llaves y relaciones.